

作并不总是那么可被一眼辨识出来，例如：

```
a[i] = a[j];           //潜在的自我赋值
```

如果 *i* 和 *j* 有相同的值，这便是个自我赋值。再看：

```
*px = *py;             //潜在的自我赋值
```

如果 *px* 和 *py* 恰巧指向同一个东西，这也是自我赋值。这些并不明显的自我赋值，是“别名”（*aliasing*）带来的结果：所谓“别名”就是“有一个以上的方法指称（指涉）某对象”。一般而言如果某段代码操作 *pointers* 或 *references* 而它们被用来“指向多个相同类型的对象”，就需考虑这些对象是否为同一个。实际上两个对象只要来自同一个继承体系，它们甚至不需声明为相同类型就可能造成“别名”，因为一个 *base class* 的 *reference* 或 *pointer* 可以指向一个 *derived class* 对象：

```
class Base { ... };
class Derived: public Base { ... };
void doSomething(const Base& rb,           //rb 和 *pd 有可能其实是同一对象
                 Derived* pd);
```

如果遵循条款 13 和条款 14 的忠告，你会运用对象来管理资源，而且你可以确定所谓“资源管理对象”在 *copy* 发生时有着正确的举措。这种情况下你的赋值操作符或许是“自我赋值安全的”（*self-assignment-safe*），不需要额外操心。然而如果你尝试自行管理资源（如果你打算写一个用于资源管理的 *class* 就得这样做），可能会掉进“在停止使用资源之前意外释放了它”的陷阱。假设你建立一个 *class* 用来保存一个指针指向一块动态分配的位图（*bitmap*）：

```
class Bitmap { ... };
class Widget {
    ...
private:
    Bitmap* pb;           //指针，指向一个从 heap 分配而得的对象
};
```

下面是 *operator=* 实现代码，表面上看起来合理，但自我赋值出现时并不安全（它也不具备异常安全性，但我们稍后才讨论这个主题）。

```
Widget&
Widget::operator=(const Widget& rhs) //一份不安全的 operator= 实现版本.
{
    delete pb;                //停止使用当前的 bitmap,
    pb = new Bitmap(*rhs.pb); //使用 rhs's bitmap 的副本（复件）。
    return *this;             //见条款 10.
}
```

这里的自我赋值问题是, operator= 函数内的 *this (赋值的目的端) 和 rhs 有可能是同一个对象。果真如此 delete 就不只是销毁当前对象的 bitmap, 它也销毁 rhs 的 bitmap。在函数末尾, Widget——它原本不该被自我赋值动作改变的——发现自己持有一个指针指向一个已被删除的对象!

欲阻止这种错误, 传统做法是藉由 operator= 最前面的一个“证同测试 (identity test)” 达到“自我赋值”的检验目的:

```
Widget& Widget::operator=(const Widget& rhs)
{
    if (this == &rhs) return *this;    //证同测试 (identity test):
                                        //如果是自我赋值, 就不做任何事。

    delete pb;
    pb = new Bitmap(*rhs.pb);
    return *this;
}
```

这样做行得通。稍早我曾经提过, 前一版 operator= 不仅不具备“自我赋值安全性”, 也不具备“异常安全性”, 这个新版本仍然存在异常方面的麻烦。更明确地说, 如果 "new Bitmap" 导致异常 (不论是因为分配时内存不足或因为 Bitmap 的 copy 构造函数抛出异常), Widget 最终会持有一个指针指向一块被删除的 Bitmap。这样的指针有害。你无法安全地删除它们, 甚至无法安全地读取它们。唯一能对它们做的安全事情是付出许多调试能量找出错误的起源。

令人高兴的是, 让 operator= 具备“异常安全性”往往自动获得“自我赋值安全”的回报。因此愈来愈多人对“自我赋值”的处理态度是倾向不去管它, 把焦点放在实现“异常安全性” (exception safety) 上。条款 29 深度探讨了异常安全性, 本条款只要你注意“许多时候一群精心安排的语句就可以导出异常安全 (以及自我赋值安全) 的代码”, 这就够了。例如以下代码, 我们只需注意在复制 pb 所指东西之前别删除 pb:

```
Widget& Widget::operator=(const Widget& rhs)
{
    Bitmap* pOrig = pb;                //记住原先的 pb
    pb = new Bitmap(*rhs.pb);          //令 pb 指向 *pb 的一个复件 (副本)
    delete pOrig;                      //删除原先的 pb
    return *this;
}
```

现在，如果 "newBitmap" 抛出异常，pb（及其栖身的那个 Widget）保持原状。即使没有证同测试（identity test），这段代码还是能够处理自我赋值，因为我们对原 bitmap 做了一份复件、删除原 bitmap、然后指向新制造的那个复件。它或许不是处理“自我赋值”的最高效办法，但它行得通。

如果你很关心效率，可以把“证同测试”（identity test）再次放回函数起始处。然而这样做之前先问问自己，你估计“自我赋值”的发生频率有多高？因为这项测试也需要成本。它会使代码变大一些（包括原始码和目标码）并导入一个新的控制流（control flow）分支，而两者都会降低执行速度。Prefetching、caching 和 pipelining 等指令的效率都会因此降低。

在 operator= 函数内手工排列语句（确保代码不但“异常安全”而且“自我赋值安全”）的一个替代方案是，使用所谓的 **copy and swap** 技术。这个技术和“异常安全性”有密切关系，所以由条款 29 详细说明。然而由于它是一个常见而够好的 operator= 撰写办法，所以值得看看其实现手法像什么样子：

```
class Widget {
...
void swap(Widget& rhs);    //交换*this 和 rhs 的数据；详见条款 29
...
};
Widget& Widget::operator=(const Widget& rhs)
{
    Widget temp(rhs);      //为 rhs 数据制作一份复件（副本）
    swap(temp);            //将*this 数据和上述复件的数据交换。
    return *this;
}
```

这个主题的另一个变奏曲乃利用以下事实：(1) 某 class 的 **copy assignment** 操作符可能被声明为“以 **by value** 方式接受实参”；(2) 以 **by value** 方式传递东西会造成一份复件/副本（见条款 20）：

```
Widget& Widget::operator=(Widget rhs) //rhs 是被传对象的一份复件（副本）
{
    //注意这里是 pass by value.
    swap(rhs);                //将*this 的数据和复件/副本的数据互换
    return *this;
}
```

我个人比较忧虑这个做法, 我认为它为了伶俐巧妙的修补而牺牲了清晰性。然而将“*copying* 动作”从函数本体内移至“函数参数构造阶段”却可令编译器有时生成更高效的代码。

请记住

- 确保当对象自我赋值时 `operator=` 有良好行为。其中技术包括比较“来源对象”和“目标对象”的地址、精心周到的语句顺序、以及 `copy-and-swap`。
- 确定任何函数如果操作一个以上的对象, 而其中多个对象是同一个对象时, 其行为仍然正确。

条款 12: 复制对象时勿忘其每一个成分

Copy all parts of an object.

设计良好之面向对象系统 (OO-systems) 会将对象的内部封装起来, 只留两个函数负责对象拷贝 (复制), 那便是带着適切名称的 *copy* 构造函数和 *copy assignment* 操作符, 我称它们为 *copying* 函数。条款 5 观察到编译器会在必要时候为我们的 `classes` 创建 *copying* 函数, 并说明这些“编译器生成版”的行为: 将被拷对象的所有成员变量都做一份拷贝。

如果你声明自己的 *copying* 函数, 意思就是告诉编译器你并不喜欢缺省实现中的某些行为。编译器仿佛被冒犯似的, 会以一种奇怪的方式回敬: 当你的实现代码几乎必然出错时却不告诉你。

考虑一个 `class` 用来表现顾客, 其中手工写出 (而非由编译器创建) *copying* 函数, 使得外界对它们的调用会被志记 (logged) 下来:

```
void logCall(const std::string& funcName);    //制造一个 log entry
class Customer {
public:
    ...
    Customer(const Customer& rhs);
    Customer& operator=(const Customer& rhs);
    ...
private:
    std::string name;
};
```

```

Customer::Customer(const Customer& rhs)
    : name(rhs.name)                //复制 rhs 的数据
{
    logCall("Customer copy constructor");
}

Customer& Customer::operator=(const Customer& rhs)
{
    logCall("Customer copy assignment operator");
    name = rhs.name;                //复制 rhs 的数据
    return *this;                   //见条款 10
}

```

这里的每一件事情看起来都很好，而实际上每件事情也的确都好，直到另一个成员变量加入战局：

```

class Date { ... };                //日期
class Customer {
public:
    ...                            //同前
private:
    std::string name;
    Date lastTransaction;
};

```

这时候既有的 *copying* 函数执行的是局部拷贝（*partial copy*）：它们的确复制了顾客的 *name*，但没有复制新添加的 *lastTransaction*。大多数编译器对此不出任何怨言——即使在最高警告级别中（见条款 53）。这是编译器对“你自己写出 *copying* 函数”的复仇行为：既然你拒绝它们为你写出 *copying* 函数，如果你的代码不完全，它们也不告诉你。结论很明显：如果你为 *class* 添加一个成员变量，你必须同时修改 *copying* 函数。（你也需要修改 *class* 的所有构造函数（见条款 4 和条款 45）以及任何非标准形式的 *operator=*（条款 10 有个例子）。如果你忘记，编译器不太可能提醒你。）

一旦发生继承，可能会造成此一主题最暗中肆虐的一个潜藏危机。试考虑：

```

class PriorityCustomer: public Customer {    //一个 derived class
public:
    ...
    PriorityCustomer(const PriorityCustomer& rhs);
    PriorityCustomer& operator=(const PriorityCustomer& rhs);
    ...
private:
    int priority;
};

```